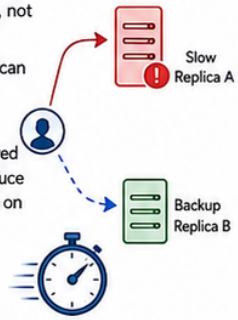
Day 31: Hedged Requests, Tail Latency, and Duplicate Suppression

Reducing tail latency by racing slow replicas carefully.

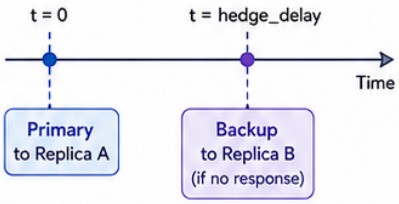
1 Why Hedged Requests Matter

- Tail latency is caused by occasional stragglers, not just average latency.
- A single slow replica can dominate end-user experience.
- Hedging sends a delayed backup request to reduce the chance of waiting on one slow path.



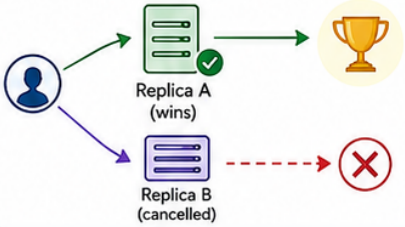
2 Hedge-Delay Strategy

- Send the primary request immediately.
- If no response by hedge_delay, send a duplicate to another replica.
- Tune hedge delay to avoid unnecessary duplicate traffic.



3 First-Success-Wins

- Earliest successful response becomes the winner.
- Slower in-flight attempts are cancelled.
- Caller sees one logical response.

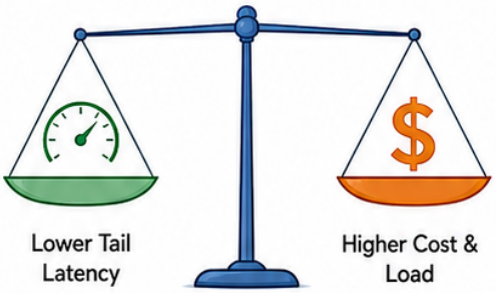


4 Duplicate Suppression & Safety

- Hedged requests can duplicate side effects on write paths.
- Use idempotency keys or prefer hedging on read-only operations.
- Cancellation should propagate quickly to reduce wasted work.

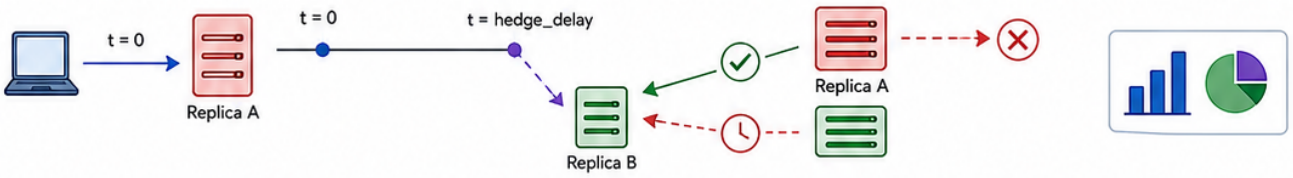
5 Trade-Offs

- Improves p95/p99 latency.
- Increases backend request volume and cost.
- Aggressive hedging can overload dependencies.
- Track hedge win rate and duplicate rate.



Sample Hedged Request Race

1. Primary request sent to Replica A at t = 0.
2. If slow, backup sent to Replica B after hedge delay.
3. First successful response wins.
4. Cancel the losing duplicate.
5. Report winner, latency saved, and duplicate count.



Winner	Latency Saved	Duplicate Count
Replica B (first success)	48 ms (example)	1 (one extra request sent)

Hedging targets tail latency, not average latency — first success wins, slower duplicates are cancelled.

System Design - Day 31

Hedged Requests, Tail Latency, and Duplicate Suppression

Prepared from Day-31 notes and sample implementation

Overview from README.md

Day 31: Hedged Requests, Tail Latency, and Duplicate Suppression

Day 31 focuses on reducing tail latency by sending backup requests when the primary call is slow.

It covers:
• hedged request fundamentals
• hedge-delay tuning
• first-success-wins behavior
• cancellation of losing duplicates
• tail-latency improvement trade-offs
• capacity and cost implications

In short, Day 31 is about improving p95/p99 latency by racing replicas carefully instead of waiting on one slow response path.

Core ideas

Why hedged requests matter

• Tail latency is often caused by occasional stragglers rather than average latency.
• A single slow replica can dominate end-user experience.
• Hedging sends a delayed backup to reduce the chance of waiting for a straggler.

Hedge-delay strategy

• Send the primary request immediately.
• If response is not back by `hedge_delay`, send a duplicate to another replica.
• Keep hedge delay large enough to avoid unnecessary duplicate traffic.

First-success-wins model

• The earliest successful response becomes the winner.
• Remaining in-flight attempts are cancelled.
• Caller sees one logical response even though multiple attempts may have been issued.

Duplicate suppression and safety

• Hedged requests can trigger duplicate side effects on write paths.
• Use idempotency keys or limit hedging to read-only operations.
• Cancellation should propagate quickly to reduce wasted work.

Trade-offs

• Hedging improves tail latency but increases backend request volume.
• Aggressive hedging can overload dependencies and increase cost.
• Rollout should be controlled with metrics on duplicate rate and win rate.

Day-31 sample: Hedged Request Race Simulator

This repository includes a small hedged-request simulator in both Python and Java.

Functional requirements

• Start primary request immediately
• Launch backup replicas after fixed hedge delay
• Choose first successful attempt as winner
• Cancel slower in-flight attempts
• Report duplicate count, winner, and latency savings vs primary-only

High-level components

• `ReplicaEndpoint`: endpoint latency and success behavior
• `AttemptRecord`: attempt start/finish metadata
• `HedgedRequestExecutor`: hedging orchestration and winner selection
• `ExecutionResult`: status, winner, completion time, and timeline

Data flow

1. Build attempt schedule (`primary` at `t=0`, backups staggered by hedge delay)
2. Track attempt finish times and find first successful completion
3. Mark winner and cancel slower in-flight attempts
4. Compute duplicate count and tail-latency savings
5. Return result timeline for debugging and interview explanation

Project structure

```
Day-31/
README.md
hedged-request-sample/
python/
hedged_request.py
demo.py
java/
HedgedRequestSystem.java
Main.java
```

Run the sample

Python

```
cd hedged-request-sample\python
python demo.py
```

Java

```
cd hedged-request-sample\java
javac Main.java HedgedRequestSystem.java
java Main
```

Interview takeaways

• Hedging targets tail latency, not average latency

- First-success-wins plus cancellation controls wasted work
- Hedge delay should be based on real latency percentiles
- Idempotency protection is mandatory for side-effecting paths
- Duplicate ratio and hedge win rate are key operational metrics

Next improvements

- Use percentile-driven adaptive hedge delay
- Cap max duplicate attempts by request class
- Add per-endpoint health scoring for hedge target selection
- Add request cost model for hedge policy optimization
- Add timeout budget integration with end-to-end deadlines

Program Listings

1. hedged-request-sample\python\hedged_request.py

```
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum


class ResultStatus(Enum):
    SUCCESS = "SUCCESS"
    FAILED = "FAILED"


@dataclass(frozen=True)
class ReplicaEndpoint:
    name: str
    latency_ms: int
    will_succeed: bool = True


@dataclass(frozen=True)
class AttemptRecord:
    endpoint_name: str
    start_ms: int
    finish_ms: int
    will_succeed: bool
    sequence: int


@dataclass(frozen=True)
class ExecutionResult:
    status: ResultStatus
    winner: str | None
    completion_ms: int
    duplicates_issued: int
    tail_saved_ms: int
    timeline: list[str]


class HedgedRequestExecutor:
    def execute(self, replicas: list[ReplicaEndpoint], hedge_delay_ms: int) -> ExecutionResult:
        if hedge_delay_ms <= 0:
            raise ValueError("hedge_delay_ms must be positive")
        if len(replicas) < 2:
            raise ValueError("replicas must contain at least two endpoints")
        if any(replica.latency_ms <= 0 for replica in replicas):
            raise ValueError("replica latency_ms must be positive")

        attempts: list[AttemptRecord] = []
        for sequence, replica in enumerate(replicas):
            start_ms = 0 if sequence == 0 else hedge_delay_ms * sequence
            attempts.append(
                AttemptRecord(
                    endpoint_name=replica.name,
                    start_ms=start_ms,
                    finish_ms=start_ms + replica.latency_ms,
                    will_succeed=replica.will_succeed,
                    sequence=sequence,
                )
            )

        successful_attempts = [attempt for attempt in attempts if attempt.will_succeed]
        winner_attempt: AttemptRecord | None = None
        winner_time: int | None = None
        if successful_attempts:
            winner_attempt = min(successful_attempts, key=lambda attempt: (attempt.finish_ms, attempt.sequence))
            winner_time = winner_attempt.finish_ms

        if winner_time is None:
            launched_attempts = attempts
        else:
            launched_attempts = [
                attempt
                for attempt in attempts
```

```
            if attempt is winner_attempt or attempt.start_ms < winner_time
        ]

    launched_keys = {
        (attempt.endpoint_name, attempt.sequence, attempt.start_ms, attempt.finish_ms)
        for attempt in launched_attempts
    }
    duplicates_issued = max(0, len(launched_attempts) - 1)
    primary_attempt = attempts[0]
    completion_ms = (
        winner_time
        if winner_time is not None
        else max(attempt.finish_ms for attempt in launched_attempts)
    )
    tail_saved_ms = (
        max(0, primary_attempt.finish_ms - winner_time)
        if winner_time is not None
        else 0
    )

    timeline = [f"request started hedge_delay={hedge_delay_ms}ms"]
    for attempt in attempts:
        key = (attempt.endpoint_name, attempt.sequence, attempt.start_ms, attempt.finish_ms)
        if key in launched_keys:
            timeline.append(
                f"{attempt.endpoint_name}: launched at t={attempt.start_ms}ms "
                f"expected_finish={attempt.finish_ms}ms"
            )
        else:
            timeline.append(
                f"{attempt.endpoint_name}: not launched "
                f"(winner decided at {winner_time}ms)"
            )

    for attempt in sorted(launched_attempts, key=lambda value: (value.finish_ms, value.sequence)):
        if winner_attempt is not None and attempt == winner_attempt:
            timeline.append(
                f"{attempt.endpoint_name}: winner success at t={attempt.finish_ms}ms"
            )
        elif winner_time is not None and attempt.finish_ms > winner_time:
            timeline.append(
                f"{attempt.endpoint_name}: cancelled at winner_time={winner_time}ms "
                f"before finish={attempt.finish_ms}ms"
            )
        elif attempt.will_succeed:
            timeline.append(
                f"{attempt.endpoint_name}: success at t={attempt.finish_ms}ms (lost race)"
            )
        else:
            timeline.append(f"{attempt.endpoint_name}: failed at t={attempt.finish_ms}ms")

    status = ResultStatus.SUCCESS if winner_attempt is not None else ResultStatus.FAILED
    timeline.append(
        f"request finished status={status.value} completion={completion_ms}ms "
        f"duplicates_issued={duplicates_issued} tail_saved={tail_saved_ms}ms"
    )
    return ExecutionResult(
        status=status,
        winner=winner_attempt.endpoint_name if winner_attempt else None,
        completion_ms=completion_ms,
        duplicates_issued=duplicates_issued,
        tail_saved_ms=tail_saved_ms,
        timeline=timeline,
    )
```

2. hedged-request-sample\python\demo.py

```
from hedged_request import HedgedRequestExecutor, ReplicaEndpoint


def print_result(title: str, replicas: list[ReplicaEndpoint], hedge_delay_ms: int) -> None:
    executor = HedgedRequestExecutor()
    result = executor.execute(replicas=replicas, hedge_delay_ms=hedge_delay_ms)
    print(title)
    print(
        f"  status={result.status.value} winner={result.winner} "
```

```
        f"completion={result.completion_ms}ms duplicates={result.duplicates_issued} "
        f"tail_saved={result.tail_saved_ms}ms"
    )
    for line in result.timeline:
        print(" ", line)


def main() -> None:
    slow_primary = [
        ReplicaEndpoint("primary-us-east", 220, True),
        ReplicaEndpoint("replica-us-west", 90, True),
        ReplicaEndpoint("replica-eu", 130, True),
    ]
    print_result("hedge beats slow primary", slow_primary, hedge_delay_ms=80)

    print()
    fast_primary = [
        ReplicaEndpoint("primary-us-east", 70, True),
        ReplicaEndpoint("replica-us-west", 90, True),
        ReplicaEndpoint("replica-eu", 110, True),
    ]
    print_result("fast primary returns before hedge launch", fast_primary, hedge_delay_ms=80)

    print()
    all_fail = [
        ReplicaEndpoint("primary-us-east", 100, False),
        ReplicaEndpoint("replica-us-west", 120, False),
        ReplicaEndpoint("replica-eu", 60, False),
    ]
    print_result("all attempts fail", all_fail, hedge_delay_ms=50)


if __name__ == "__main__":
    main()
```

3. hedged-request-sample\java\HedgedRequestSystem.java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class HedgedRequestSystem {
    public enum ResultStatus {
        SUCCESS,
        FAILED
    }

    public static final class ReplicaEndpoint {
        private final String name;
        private final int latencyMs;
        private final boolean willSucceed;

        public ReplicaEndpoint(String name, int latencyMs, boolean willSucceed) {
            this.name = name;
            this.latencyMs = latencyMs;
            this.willSucceed = willSucceed;
        }

        public String getName() {
            return name;
        }

        public int getLatencyMs() {
            return latencyMs;
        }

        public boolean willSucceed() {
            return willSucceed;
        }
    }

    public static final class AttemptRecord {
        private final String endpointName;
        private final int startMs;
        private final int finishMs;
        private final boolean willSucceed;
```

```
    private final int sequence;

    public AttemptRecord(String endpointName, int startMs, int finishMs, boolean willSucceed, int sequence) {
        this.endpointName = endpointName;
        this.startMs = startMs;
        this.finishMs = finishMs;
        this.willSucceed = willSucceed;
        this.sequence = sequence;
    }

    public String getEndpointName() {
        return endpointName;
    }

    public int getStartMs() {
        return startMs;
    }

    public int getFinishMs() {
        return finishMs;
    }

    public boolean willSucceed() {
        return willSucceed;
    }

    public int getSequence() {
        return sequence;
    }
}

public static final class ExecutionResult {
    private final ResultStatus status;
    private final String winner;
    private final int completionMs;
    private final int duplicatesIssued;
    private final int tailSavedMs;
    private final List<String> timeline;

    public ExecutionResult(
        ResultStatus status,
        String winner,
        int completionMs,
        int duplicatesIssued,
        int tailSavedMs,
        List<String> timeline
    ) {
        this.status = status;
        this.winner = winner;
        this.completionMs = completionMs;
        this.duplicatesIssued = duplicatesIssued;
        this.tailSavedMs = tailSavedMs;
        this.timeline = timeline;
    }

    public ResultStatus getStatus() {
        return status;
    }

    public String getWinner() {
        return winner;
    }

    public int getCompletionMs() {
        return completionMs;
    }

    public int getDuplicatesIssued() {
        return duplicatesIssued;
    }

    public int getTailSavedMs() {
        return tailSavedMs;
    }

    public List<String> getTimeline() {
        return timeline;
```

```
    }
}

public static final class HedgedRequestExecutor {
    public ExecutionResult execute(List<ReplicaEndpoint> replicas, int hedgeDelayMs) {
        if (hedgeDelayMs <= 0) {
            throw new IllegalArgumentException("hedgeDelayMs must be positive");
        }
        if (replicas == null || replicas.size() < 2) {
            throw new IllegalArgumentException("replicas must contain at least two endpoints");
        }

        List<AttemptRecord> attempts = new ArrayList<>();
        for (int i = 0; i < replicas.size(); i++) {
            ReplicaEndpoint replica = replicas.get(i);
            if (replica.getLatencyMs() <= 0) {
                throw new IllegalArgumentException("replica latencyMs must be positive");
            }
            int startMs = i == 0 ? 0 : hedgeDelayMs * i;
            attempts.add(
                new AttemptRecord(
                    replica.getName(),
                    startMs,
                    startMs + replica.getLatencyMs(),
                    replica.willSucceed(),
                    i
                )
            );
        }

        AttemptRecord winner = null;
        for (AttemptRecord attempt : attempts) {
            if (!attempt.willSucceed()) {
                continue;
            }
            if (winner == null) {
                winner = attempt;
                continue;
            }
            if (attempt.getFinishMs() < winner.getFinishMs()) {
                winner = attempt;
            } else if (
                attempt.getFinishMs() == winner.getFinishMs()
                    && attempt.getSequence() < winner.getSequence()
            ) {
                winner = attempt;
            }
        }

        Integer winnerTime = winner == null ? null : winner.getFinishMs();
        List<AttemptRecord> launchedAttempts = new ArrayList<>();
        if (winnerTime == null) {
            launchedAttempts.addAll(attempts);
        } else {
            for (AttemptRecord attempt : attempts) {
                if (attempt == winner || attempt.getStartMs() < winnerTime) {
                    launchedAttempts.add(attempt);
                }
            }
        }

        int duplicatesIssued = Math.max(0, launchedAttempts.size() - 1);
        AttemptRecord primary = attempts.get(0);
        int completionMs = winnerTime != null
            ? winnerTime
            : launchedAttempts.stream().mapToInt(AttemptRecord::getFinishMs).max().orElse(0);
        int tailSavedMs = winnerTime == null ? 0 : Math.max(0, primary.getFinishMs() - winnerTime);

        List<String> timeline = new ArrayList<>();
        timeline.add("request started hedge_delay=" + hedgeDelayMs + "ms");
        for (AttemptRecord attempt : attempts) {
            boolean launched = launchedAttempts.contains(attempt);
            if (launched) {
                timeline.add(
                    attempt.getEndpointName()
                        + ": launched at t="
                        + attempt.getStartMs()
```

```
                        + "ms expected_finish="
                        + attempt.getFinishMs()
                        + "ms"
                    );
                } else {
                    timeline.add(
                        attempt.getEndpointName()
                            + ": not launched (winner decided at "
                            + winnerTime
                            + "ms)"
                    );
                }
            }

            launchedAttempts.sort(
                Comparator
                    .comparingInt(AttemptRecord::getFinishMs)
                    .thenComparingInt(AttemptRecord::getSequence)
            );
            for (AttemptRecord attempt : launchedAttempts) {
                if (winner != null && attempt == winner) {
                    timeline.add(attempt.getEndpointName() + ": winner success at t=" + attempt.getFinishMs() + "m
                } else if (winnerTime != null && attempt.getFinishMs() > winnerTime) {
                    timeline.add(
                        attempt.getEndpointName()
                            + ": cancelled at winner_time="
                            + winnerTime
                            + "ms before finish="
                            + attempt.getFinishMs()
                            + "ms"
                    );
                } else if (attempt.willSucceed()) {
                    timeline.add(attempt.getEndpointName() + ": success at t=" + attempt.getFinishMs() + "ms (lost
                } else {
                    timeline.add(attempt.getEndpointName() + ": failed at t=" + attempt.getFinishMs() + "ms");
                }
            }

            ResultStatus status = winner == null ? ResultStatus.FAILED : ResultStatus.SUCCESS;
            timeline.add(
                "request finished status="
                    + status
                    + " completion="
                    + completionMs
                    + "ms duplicates_issued="
                    + duplicatesIssued
                    + " tail_saved="
                    + tailSavedMs
                    + "ms"
            );

            return new ExecutionResult(
                status,
                winner == null ? null : winner.getEndpointName(),
                completionMs,
                duplicatesIssued,
                tailSavedMs,
                timeline
            );
        }
    }
}
```

4. hedged-request-sample\java\Main.java

```
import java.util.List;

public class Main {
    private static void printResult(String title, List<HedgedRequestSystem.ReplicaEndpoint> replicas, int hedgeDel
        HedgedRequestSystem.HedgedRequestExecutor executor = new HedgedRequestSystem.HedgedRequestExecutor();
        HedgedRequestSystem.ExecutionResult result = executor.execute(replicas, hedgeDelayMs);

        System.out.println(title);
        System.out.println(
            "  status="
                + result.getStatus()
```

```
                + " winner="
                + result.getWinner()
                + " completion="
                + result.getCompletionMs()
                + "ms duplicates="
                + result.getDuplicatesIssued()
                + " tail_saved="
                + result.getTailSavedMs()
                + "ms"
        );
        for (String line : result.getTimeline()) {
            System.out.println("  " + line);
        }
    }

    public static void main(String[] args) {
        List<HedgedRequestSystem.ReplicaEndpoint> slowPrimary = List.of(
            new HedgedRequestSystem.ReplicaEndpoint("primary-us-east", 220, true),
            new HedgedRequestSystem.ReplicaEndpoint("replica-us-west", 90, true),
            new HedgedRequestSystem.ReplicaEndpoint("replica-eu", 130, true)
        );
        printResult("hedge beats slow primary", slowPrimary, 80);

        System.out.println();
        List<HedgedRequestSystem.ReplicaEndpoint> fastPrimary = List.of(
            new HedgedRequestSystem.ReplicaEndpoint("primary-us-east", 70, true),
            new HedgedRequestSystem.ReplicaEndpoint("replica-us-west", 90, true),
            new HedgedRequestSystem.ReplicaEndpoint("replica-eu", 110, true)
        );
        printResult("fast primary returns before hedge launch", fastPrimary, 80);

        System.out.println();
        List<HedgedRequestSystem.ReplicaEndpoint> allFail = List.of(
            new HedgedRequestSystem.ReplicaEndpoint("primary-us-east", 100, false),
            new HedgedRequestSystem.ReplicaEndpoint("replica-us-west", 120, false),
            new HedgedRequestSystem.ReplicaEndpoint("replica-eu", 60, false)
        );
        printResult("all attempts fail", allFail, 50);
    }
}
```